

Reviewer Pack — Matroid Rank Deficit in Monotone DNF for k-CLIQUE

Entry b0d510339340 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 11:53:19 UTC

Matroid Rank Deficit in Monotone DNF for k-CLIQUE

Entry ID: b0d510339340 Verdict: INCONCLUSIVE Recorded: 2026-05-19 11:53:19 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

For any monotone DNF formula computing k-CLIQUE on n vertices, the maximum number of pairwise disjoint clauses is $\Omega(n)$, while for any DNF of size $\text{poly}(n)$, this number is $O(\log n)$.

Rationale (proposer’s reasoning):

Matroid rank functions are submodular and capture combinatorial independence, making them natural for analyzing DNF structure. The conjecture links the inherent complexity of k-CLIQUE to matroid-theoretic parameters, potentially revealing structural barriers for monotone circuits.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6c0817233f1bddca

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, k):
        clauses = set()
        for _ in range(k):
            clause = {random.randint(1, n) for _ in range(random.randint(1, n))}
            clauses.add(tuple(sorted(clause)))
        return clauses

    def is_k_clique(DNF, n, k):
        vertices = set(range(1, n + 1))
        for clause in DNF:
            if len(vertices.intersection(clause)) < k:
                return False
        return True

    def max_pairwise_disjoint_clauses(DNF):
        clauses = list(DNF)
        disjoint_clauses = []
        while clauses:
            selected_clause = clauses[0]
            disjoint_clauses.append(selected_clause)
            clauses = [c for c in clauses if not c.intersection(selected_clause)]
        return len(disjoint_clauses)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        DNF = generate_dnf(n, k=5)
        if is_k_clique(DNF, n, 5):
            metric_value = max_pairwise_disjoint_clauses(DNF)
            conjecture_holds = metric_value >= n
            counterexample = "" if conjecture_holds else "k-CLIQUE instance"
        else:
            metric_value = max_pairwise_disjoint_clauses(DNF)
            conjecture_holds = metric_value <= math.log(n, 2)
```

```

        counterexample = "" if conjecture_holds else "random DNF"

    results.append({
        "metric_name": "max_disjoint_clauses",
        "metric_value": metric_value,
        "instances_tested": len(DNF),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    })

    return {
        "seed": seed,
        "results": results
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + list(range(50, 100))

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.extend(result["results"])

    mean_value = sum(r["metric_value"] for r in all_results) / len(all_results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in all_results) / len(all_results))
    support_fraction = sum(1 for r in all_results if r["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(not r["conjecture_holds"] for r in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b5f6a9c.py", line 77, in <module>

```

    result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b5f6a9c.py", line 54, in run_trial

```

    metric_value = max_pairwise_disjoint_clauses(DNF)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b5f6a9c.py", line 41, in max_pairwise_

```

    clauses = [c for c in clauses if not c.intersection(selected_clause)]
    ~~~~~
```

AttributeError: 'tuple' object has no attribute 'intersection'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to AttributeError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the 'tuple' object 'intersection' error in the max_pairwise_disjoint_clauses function

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	56942
2	propose	ollama_remote	qwen3:8b	0	0	60379
3	preregistration	ollama_remote	qwen3:8b	0	0	27774
4	novelty	ollama_remote	qwen3:8b	0	0	24060
5	novelty	ollama_remote	qwen3:8b	0	0	22420
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12415
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7856
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8249
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8969
10	judge	ollama_remote	qwen3:8b	0	0	21202

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 250267 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b0d510339340.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0d510339340.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b0d510339340.tar.gz` (if generated)